

codex-token-monitor

Stdout-Anzeige für Token- und Kontext-Daten der aktuell laufenden Codex-Session. Liest direkt die Rollout-JSONL-Datei unter `$CODEX_HOME/sessions/...` — kein Patchen von Codex nötig, keine Abhängigkeit auf interne Codex-Crates. Nur die JSON-Felder, die das Tool nutzt, sind als minimaler Serde-Typ nachgebaut; unbekannte Felder werden ignoriert, damit Codex-Updates nichts brechen.

Schnellinstallation (ohne Bauen)

Vorgefertigte Binaries gibt es auf der [Releases-Seite](#).

Statisch verlinkte Variante (empfohlen — läuft auf jedem x86_64-Linux/WSL2 ohne Abhängigkeiten):

```
mkdir -p ~/.local/bin
curl -sSL https://github.com/Robbty/codex-token-monitor/releases/latest/download/codex-tokens-x86_64-linux-musl \
  -o ~/.local/bin/codex-tokens
chmod +x ~/.local/bin/codex-tokens
codex-tokens --version
```

Falls `~/.local/bin` nicht im `$PATH` liegt, in `~/.bashrc` ergänzen:

```
export PATH="$HOME/.local/bin:$PATH"
```

Integrität prüfen (optional, aber empfohlen):

```
curl -sSL https://github.com/Robbty/codex-token-monitor/releases/latest/download/SHA256SUMS \
  | grep codex-tokens-x86_64-linux-musl \
  | sha256sum -c -
```

Für ältere Distros bzw. wenn die dynamische Variante reicht, die `-gnu`-Datei verwenden — Voraussetzung: `glibc ≥ 2.35`.

Bauen

```
cd /pfad/zu/codex-token-monitor
cargo build --release
# Binary: target/release/codex-tokens
```

Das Binary ist dynamisch gegen die `glibc` des Build-Systems gelinkt — siehe [Portabilität](#) für die Variante, die auf jedem Linux läuft.

Installation

Damit `codex-tokens` aus jedem Terminal aufrufbar ist, das Binary in ein Verzeichnis im `$PATH` legen. Übliche Optionen:

Variante 1 — `~/.local/bin` (XDG-Standard, empfohlen):

```
mkdir -p ~/.local/bin
install -m 755 target/release/codex-tokens ~/.local/bin/

# Prüfen, ob ~/.local/bin im PATH steht:
echo "$PATH" | tr ':' '\n' | grep -q "$HOME/.local/bin" \
  && echo "PATH ok" \
  || echo 'export PATH="$HOME/.local/bin:$PATH" # in ~/.bashrc ergänzen'
```

Variante 2 — `~/bin` (klassisch):

```
mkdir -p ~/bin
install -m 755 target/release/codex-tokens ~/bin/
# Bei Bedarf in ~/.bashrc ergänzen:
# export PATH="$HOME/bin:$PATH"
```

Variante 3 — /usr/local/bin (systemweit, benötigt sudo):

```
sudo install -m 755 target/release/codex-tokens /usr/local/bin/
```

Variante 4 — direkt mit Cargo:

```
cargo install --path /pfad/zu/codex-token-monitor
# Binary landet in ~/.cargo/bin/codex-tokens (in $PATH, wenn Rust regulär installiert wurde)
```

Hinweis: /bin ist auf Linux meist ein Symlink auf /usr/bin und für System-Tools reserviert — eigene Binaries gehören dort nicht hin. Nimm stattdessen Variante 1, 2 oder 3.

Test nach der Installation:

```
which codex-tokens
codex-tokens --version
```

Verwendung

codex-tokens [OPTIONS]

```
--thread <UUID>      An eine konkrete Session binden (Thread-/Session-ID).
--cwd [PATH]          An die Session binden, deren cwd zu PATH passt (Default: $PWD).
--codex-home DIR      Überschreibt CODEX_HOME (Default: $CODEX_HOME oder ~/.codex).
-f, --follow          Folgt der Rollout-Datei und gibt bei jedem TokenCount-Event eine neue Mome
--json               Einzeiliges JSON statt KEY=VALUE.
--locate             Gibt den ermittelten Rollout-Pfad aus und beendet sich.
--wait               Wartet, bis eine passende Rollout-Datei erscheint, statt sofort abzubreche
                    Damit kann der Monitor vor Codex gestartet werden.
--wait-timeout SECS  Sekunden-Limit für --wait (Default: unbegrenzt).
--all               Multi-Session: mit --cwd ALLE aktiven passenden Sessions gleichzeitig
                    verfolgen (statt nur der neuesten). Jeder Block wird mit
                    "=== session <uuid> ===" markiert.
--max-age MINUTES    Nur mit --all: Rollouts, die länger nicht beschrieben wurden,
                    gelten als nicht-aktiv (Default: 5).
--watch-new          Nur mit --all --follow: scannt alle 5 s nach neu aufgetauchten Rollouts
                    und hängt sie live in den Stream.
-h, --help / -V, --version
```

Standardauswahl: die zuletzt geschriebene Rollout-Datei (neueste mtime).

Schnellstart: Live-Anzeige neben einer Codex-Session

Zwei Terminals, beide im selben Projektverzeichnis. Die Reihenfolge ist egal — dank --wait darf der Monitor vor oder nach Codex starten.

Terminal 1 — Monitor:

```
cd /pfad/zu/deinem-projekt
codex-tokens --cwd --follow --wait
```

Terminal 2 — Codex:

```
cd /pfad/zu/deinem-projekt
```

codex

In Terminal 1 läuft ab jetzt ein Dauerstream: nach jedem abgeschlossenen Codex-Turn erscheint ein neuer Snapshot, getrennt durch eine `---`-Zeile.

Beenden: `Strg+C` im Monitor-Terminal.

Mehrere Codex-Sessions im selben Verzeichnis

Standardmäßig betrachtet `codex -tokens` nur **eine** Session pro Verzeichnis (die jüngste). Wenn du mehrere Codex-Instanzen parallel im selben Projektverzeichnis laufen lässt — etwa in verschiedenen `tmux`-Panels oder Terminals — kannst du sie alle gemeinsam erfassen:

```
codex-tokens --cwd --all --follow
```

Jede Session wird in der Ausgabe durch einen eigenen Block-Header sichtbar abgegrenzt:

```
=== session 019e2b32-2d0a-7951-802a-86f2f48d1b5c ===
session_id=019e2b32-2d0a-7951-802a-86f2f48d1b5c
session_cwd=/pfad/zum/projekt
percent_left=95
...
---
=== session 019e1716-ae2-7402-8806-2623d765c67c ===
session_id=019e1716-ae2-7402-8806-2623d765c67c
session_cwd=/pfad/zum/projekt
percent_left=42
...
---
```

Im JSON-Modus (`--json --all`) wird stattdessen NDJSON ausgegeben: eine JSON-Zeile pro Session und Update, ideal für `jq`-Pipelines.

Was zählt als "aktive" Session?

Per Default werden Rollouts ignoriert, deren letzte Änderung länger als **5 Minuten** zurückliegt. Damit fallen alte Sessions desselben Verzeichnisses automatisch raus. Das Fenster ist konfigurierbar:

```
codex-tokens --cwd --all --max-age 30    # 30 min - großzügiger
codex-tokens --cwd --all --max-age 1     # 1 min - nur ganz frische
```

Soll nichts gefiltert werden (z. B. für eine Archiv-Auswertung), kann ein sehr hoher Wert verwendet werden:

```
codex-tokens --cwd --all --max-age 99999
```

Neue Sessions während des Laufs erkennen

Standardmäßig folgt der Monitor nur den Sessions, die beim Start vorhanden waren. Mit `--watch-new` scannt er zusätzlich alle 5 Sekunden das Sessions-Verzeichnis auf neu aufgetauchte Rollouts:

```
codex-tokens --cwd --all --follow --watch-new --wait
```

Damit kannst du den Monitor *vor* Codex starten und beliebig viele Codex-Instanzen nachträglich hinzustarten — sie erscheinen automatisch im Stream.

Filtern in Bash

Nur eine bestimmte Session aus dem Multi-Stream herauspicken:

```
SESSION=019e2b32-2d0a-7951-802a-86f2f48d1b5c
codex-tokens --cwd --all --follow \
| awk -v s===" session $SESSION ===" '
    $0 == s {p=1}
    p {print}
    p && /^---$/ {p=0}
    ,
```

Mit JSON-Output und jq:

```
codex-tokens --cwd --all --json --follow \
| jq -c --arg s "$SESSION" 'select(.session_id == $s)'
```

Aggregierte Übersicht aller aktiven Sessions:

```
codex-tokens --cwd --all --json \
| jq -s 'map({id: .session_id, left: .percent_left, used: .tokens_in_context})'
```

Aufruf innerhalb der Codex-TUI

Manchmal will man den Token-Stand sehen, ohne ein zweites Terminal zu öffnen. Codex kann das Kommando in der laufenden TUI selbst ausführen.

Variante A — Direkt ausführen mit !-Präfix (empfohlen):

In der Eingabezeile tippen:

```
!codex-tokens --cwd
```

Das ! führt das Kommando sofort aus, ohne das Modell zu befragen. Die Ausgabe landet direkt im Chat-Verlauf, kostet kein Modell-Token und bricht den laufenden Codex-Gedankengang nicht ab.

Variante B — Codex selbst aufrufen lassen:

Wenn das Modell die Zahlen in seine Antwort einbeziehen soll, einfach fragen:

Führe `codex-tokens --cwd` aus und sag mir, wie viel Kontext noch frei ist.

Codex führt das Kommando in einem normalen Tool-Call aus und kommentiert das Ergebnis (z. B. „*nur noch 8 % frei, vielleicht / compact ausführen?*“). Kostet einen Turn.

Warnung: --follow / -f niemals in der TUI verwenden!

`codex-tokens --follow` läuft endlos. Innerhalb der Codex-TUI würde dieses Kommando **die laufende Eingabe komplett blockieren**, weil Codex auf das Ende des Shell-Aufrufs wartet, das niemals kommt. Verwende `--follow` **ausschließlich** in einem separaten Terminal.

In der TUI nur Snapshot-Aufrufe (ohne -f):

```
!codex-tokens --cwd           # ok
!codex-tokens --cwd --json    # ok
!codex-tokens --cwd --follow  # NICHT verwenden!
```

Falls doch einmal -f in der TUI passiert ist

Wenn Codex auf einem festhängenden `codex-tokens --follow` wartet, lässt sich das wieder lösen:

1. **Esc drücken** — Codex bricht das laufende Tool/Kommando ab. Die TUI bleibt offen, du kannst normal weitermachen.

2. Falls Esc nichts hilft: im *anderen* Terminal das Kommando killen:

```
bash pkill -INT codex-tokens
```

1. Letzte Option — Codex komplett beenden: zweimal Strg+C drücken (Codex zeigt nach dem ersten Druck den Hinweis „Press again to quit“). Das schließt die TUI sauber; der nächste codex-Aufruf öffnet eine neue Session.

Ausgabeformat (KEY=VALUE, Default)

```
session_id=019e218d-...
session_cwd=/pfad/zu/deinem-projekt
context_window=258400
percent_left=79
percent_used=21
tokens_in_context=63437      # Aktuelle Belegung des Kontextfensters (last_token_usage.total_t
session_total_tokens=1484129 # Kumulierter Session-Verbrauch
total_input_tokens=...
total_cached_input_tokens=...
total_output_tokens=...
total_reasoning_output_tokens=...
total_tokens=...
total_blended=...
last_input_tokens=...
last_cached_input_tokens=...
last_output_tokens=...
last_reasoning_output_tokens=...
last_total_tokens=...
last_blended=...
rate_limit_id=codex
plan_type=prolite
primary_used_percent=10.0
primary_window_minutes=300
primary_resets_at=1778685248
secondary_used_percent=3.0
secondary_window_minutes=10080
secondary_resets_at=1779202534
---
```

Der Trenner - - - schließt einen Block ab — relevant im - - follow-Modus, wo mehrere Snapshots hintereinander geschrieben werden.

Bash-Beispiele

Direkt-Auswertung per eval (nur Variablen ohne unsichere Zeichen):

```
eval "$(codex-tokens | grep -E '^(percent_left|tokens_in_context|context_window)=')"
```

```
echo "Frei:    ${percent_left}%"
```

```
echo "Belegt:  ${tokens_in_context} / ${context_window} Token"
```

Einzelner Wert via awk:

```
PCT=$(codex-tokens | awk -F= '/^percent_left=/ {print $2}')
```

```
echo "Noch $PCT % Kontext frei"
```

JSON-Pipeline mit jq:

```
codex-tokens --json | jq '{left: .percent_left, used: .tokens_in_context}'
```

Live-Logfile schreiben:

```
codex-tokens --follow --json >> ~/codex-tokens.log &
```

Spezifische Session per UUID (aus TUI ablesbar):

```
codex-tokens --thread 019e218d-4998-7183-9bb0-e48103c4fe30
```

An die Session im aktuellen Projektverzeichnis koppeln:

```
codex-tokens --cwd
```

Wie wird die richtige Session erkannt?

Drei Modi, in dieser Priorität:

1. `--thread <UUID>` — Dateiname endet auf `-<UUID>.jsonl`. Deterministisch.
2. `--cwd [PATH]` — liest die erste Zeile (`session_meta`) jeder Rollout-Datei und nimmt die neueste, deren cwd zum angegebenen Pfad passt.
3. Ohne Option: die jüngste Datei nach Modifikationszeit. Bei mehreren parallel laufenden Codex-Sessions ggf. ambig — dann lieber `--thread`, `--cwd` oder den Multi-Session-Modus ([siehe oben](#)) verwenden.

Portabilität und statischer Build

Der Standard-Build mit `cargo build --release` erzeugt ein dynamisch gegen glibc gelinktes Binary. Es läuft auf jedem **x86_64 Linux** mit gleicher oder neuerer glibc als das Build-System (typisch: alle aktuellen Distros, auch WSL2 mit Standard-Ubuntu). Auf älteren Systemen (Ubuntu 20.04, CentOS 7), auf Alpine/musl-Distros oder unter ARM funktioniert es so nicht.

Statisch verlinktes Binary für maximale Portabilität

Da das Tool reines Rust ohne C-Abhängigkeiten ist, lässt sich mit dem musl-Target ein **vollständig statisches** Binary bauen, das auf praktisch jedem x86_64-Linux läuft — unabhängig von glibc-Version, Distribution und auch in minimalen Containern wie scratch oder Alpine.

```
# Einmalig: musl-Target installieren
rustup target add x86_64-unknown-linux-musl

# Statisches Binary bauen
cargo build --release --target x86_64-unknown-linux-musl

# Ergebnis (komplett portabel, ~1 MB):
# target/x86_64-unknown-linux-musl/release/codex-tokens
```

Auf manchen Distros (Ubuntu, Debian) wird zusätzlich das System-Paket `musl-tools` benötigt:

```
sudo apt install musl-tools      # Debian/Ubuntu
sudo dnf install musl-gcc        # Fedora
```

Test, dass das Resultat tatsächlich statisch ist:

```
file target/x86_64-unknown-linux-musl/release/codex-tokens
# → "statically linked"
ldd target/x86_64-unknown-linux-musl/release/codex-tokens
# → "not a dynamic executable"
```

Andere Architekturen

Zielsystem	Target-Triple
x86_64 Linux glibc	x86_64-unknown-linux-gnu
x86_64 Linux musl	x86_64-unknown-linux-musl
ARM64 Linux (RPi 4+)	aarch64-unknown-linux-gnu
ARM64 Linux musl	aarch64-unknown-linux-musl

Cross-Kompilation funktioniert mit `rustup target add <triple>` plus den passenden Linker (z. B. `gcc-aarch64-linux-gnu`). Für die meisten Fälle ist die musl-Variante `x86_64` die richtige Wahl.

Was sicher nicht geht

- Direkt auf **Windows** (außer in WSL2) — Windows-Native bräuchte einen Windows-Build (`x86_64-pc-windows-gnu/-msvc`).
- Auf **macOS** — bräuchte einen macOS-Build.

Beides ginge prinzipiell, aber Codex selbst läuft typischerweise unter Linux/WSL2/macOS, daher ist dort dann auch das passende Build-Target anzuziehen.

Robustheit gegenüber Codex-Updates

Die Datenstrukturen in `src/protocol.rs` sind ein minimaler Serde-Spiegel der JSON-Form auf Disk. Codex schreibt diese Form sowohl in Rollout-Dateien als auch über sein App-Server-Protokoll (TS-/JsonSchema-exportiert) und hält sie bewusst stabil. Solange die Feldnamen `total_token_usage`, `last_token_usage`, `model_context_window`, `input_tokens`, `cached_input_tokens`, `output_tokens`, `reasoning_output_tokens`, `total_tokens`, `rate_limits` bestehen, läuft das Tool ohne Anpassung weiter — neue/zusätzliche Felder werden ignoriert.